# 📖 DAY 2: COMPLETE TOPIC-BY-TOPIC MASTER STUDY GUIDE

## Traditional Machine Learning & Parallel Computing using Intel oneAPI

**Course Code:** `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
**Lead Trainer:** Shivam Gupta | **Target Session:** Day 2 (Strictly 3 Hours / 180 Minutes)
**Primary Reference:** `Intel & NEC - High Performance Computing.pdf` (Page 2)

---

## 🧭 MODULE ROADMAP

- **Module 1**: Parallel Computing Fundamentals & Amdahl's Law
- **Module 2**: Shared-Memory Multithreading with OpenMP
- **Module 3**: Cache Mechanics, MESI Protocol & False Sharing
- **Module 4**: Distributed-Memory Systems with MPI
- **Module 5**: Data Decomposition & Synchronization
- **Module 6**: Intel AI Analytics Toolkit Architecture
- **Module 7**: Intel Extension for Scikit-learn ( `sklearnex` ) & oneDAL
- **Module 8**: Performance Benchmarking & Scaling Math

---

## Module 1: Parallel Computing Fundamentals & Amdahl's Law

### 1.1 The Physical Wall of Serial Computing

For decades, processor performance followed **Dennard Scaling**: as transistors shrank, clock frequencies increased without increasing power density. However, around 2005, silicon hit the **Power Wall** ($P = C \cdot V^2 \cdot f$). Operating processors above 4 to 5 GHz generates unmanageable heat dissipation. To continue increasing computational throughput, semiconductor architects shifted from single fast cores to **multi-core processors** (e.g., 32 to 128 cores on modern Intel Xeon Scalable processors).

### 1.2 The Serial Bottleneck & Amdahl's Law

A serial algorithm executes one instruction at a time on a single CPU core, leaving all other cores idle. **Amdahl's Law** defines the upper bound of theoretical speedup:

$$\text{Speedup}(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Where:

- $P$ = Fraction of the program that can be parallelized ($0 \leq P \leq 1$).
- $(1 - P)$ = Serial fraction that must execute sequentially.

- $N$ = Number of parallel processing units.

**Key Insight**: As $N \to \infty$, $\text{Speedup}_{\max} \to \frac{1}{1-P}$. If 10% of an application is serial ($P = 0.90$), the speedup cannot exceed $\frac{1}{0.10} = 10\times$, regardless of whether you run on 64 cores or 64,000 cores!

### 1.3 Gustafson-Barsis Law (Scaled Speedup)

While Amdahl's Law assumes a fixed problem size, **Gustafson's Law** states that as computing power expands, developers increase the problem size to utilize the extra memory:

$$\text{Speedup}(N) = N - \alpha(N - 1)$$

Where $\alpha = (1 - P)$. On large clusters, data size scales with processor count, enabling supercomputers to achieve near-linear speedups on massive datasets.

---

## Module 2: Shared-Memory Multithreading with OpenMP

### 2.1 The Fork-Join Execution Paradigm

OpenMP (Open Multi-Processing) is an API for shared-memory parallel programming in C, C++, and Fortran:

1. **Master Thread**: The application starts as a single sequential thread.
2. **Fork**: Upon encountering `#pragma omp parallel`, the master thread spawns a team of worker threads across CPU cores.
3. **Work-Sharing**: Work is distributed among threads (e.g., `#pragma omp for`).
4. **Join / Barrier**: Threads synchronize at the end of the parallel block and merge back into the master thread.

### 2.2 Critical Pragmas & Reduction

- **Race Condition**: Occurs when multiple threads update a shared variable concurrently without synchronization.
- **The Reduction Clause**: Resolves race conditions without lock contention:

  ```
  double total = 0.0;
  #pragma omp parallel for reduction(+:total)
  for (size_t i = 0; i < N; ++i) {
      total += data[i];
  }
  ```

  Each thread accumulates into a private register/L1 cache location. OpenMP combines the private values into `total` at the end using an efficient binary tree reduction.

### 2.3 Static vs. Dynamic Scheduling

- `schedule(static, chunk)`: Slices loop iterations into fixed chunks assigned round-robin at start time. Zero runtime scheduling overhead; ideal for balanced workloads.
- `schedule(dynamic, chunk)`: Idle threads request chunks from a shared queue. Ideal for irregular workloads (e.g., ray tracing, sparse matrices).

---

## Module 3: Cache Mechanics, MESI Protocol & False Sharing

### 3.1 The 64-Byte Cache Line Reality

Processors never read or write single bytes to main memory; they transfer data in fixed **64-byte Cache Lines**. When multi-threading:

- **True Sharing**: Multiple threads read the same shared data (read-only is fast and cache-coherent).
- **False Sharing**: Two threads write to **different variables** that happen to reside within the **SAME 64-byte cache line**.

### 3.2 The MESI Cache Coherence Protocol

1. Core 0 modifies Variable A $\rightarrow$ Cache line in Core 0 marked `Modified (M)`.
2. Hardware bus issues an Invalidation Request to all other cores.
3. Core 1 attempting to write Variable B finds its cache line marked `Invalid (I)`!
4. Core 1 must stall its execution pipeline and reload the entire 64-byte cache line from L3 cache or DRAM, jumping latency from **1 ns to 80 ns**!

### 3.3 Eliminating False Sharing

Align thread-local data structures to 64-byte boundaries:

```
struct alignas(64) ThreadData {
    long long counter;
};
```

---

## Module 4: Distributed-Memory Systems with MPI

### 4.1 The Message Passing Interface Standard

When data exceeds a single server's RAM, execution must span multiple physical compute nodes. Because nodes do not share physical memory, they communicate over high-speed networks (InfiniBand, RoCE) using **MPI**:

- **Communicator**: A collection of communicating processes (`MPI_COMM_WORLD`).
- **Rank**: The unique rank ID $(0, 1, ...N-1)$ of a process.
- **Size**: Total number of processes launched via `mpirun -n <size>`.

### 4.2 Point-to-Point vs. Collective Communication

- **Point-to-Point**:
  - `MPI_Send(&buf, count, type, dest, tag, comm)`
  - `MPI_Recv(&buf, count, type, src, tag, comm, &status)`
- **Collective Operations**:
  - `MPI_Bcast`: One process broadcasts data to all processes.
  - `MPI_Scatter`: Divides an array among all ranks.
  - `MPI_Gather`: Gathers sub-arrays from all ranks back into the root.
  - `MPI_Reduce`: Gathers data and applies a reduction operator (`MPI_SUM`, `MPI_MAX`).
  - `MPI_Allreduce`: Reduces data and delivers the final result to **all** ranks.

## Module 5: Data Decomposition & Synchronization

### 5.1 1D and 2D Domain Partitioning

- **1D Block Decomposition**: Array of size $N$ split into contiguous blocks of size $N/P$.
- **2D Block Decomposition**: Matrices partitioned into sub-grids of size $(M/P_x) \times (N/P_y)$. Minimizes perimeter-to-area communication volume for stencil computations.

### 5.2 Halo Exchange (Ghost Cells)

During iterative grid simulations, each rank must exchange outer boundary rows/columns with neighboring ranks before computing the next time step. Handled with non-blocking `MPI_Isend` and `MPI_Irecv` to overlap communication with interior grid computation.

---

## Module 6: Intel AI Analytics Toolkit Architecture

### 6.1 The AI Kit Ecosystem

The **Intel AI Analytics Toolkit (AI Kit)** provides drop-in acceleration for data science workflows:

1. **Intel Distribution for Python**: High-performance Python binaries linked against Intel runtime libraries.
2. **Intel-Optimized NumPy**: Replaces standard OpenBLAS with **Intel oneMKL**, providing auto-vectorized BLAS Level 3 matrix routines.
3. **Accelerated Pandas via Modin**: Distributes Pandas Dataframe queries across all available CPU cores.

---

## Module 7: Intel Extension for Scikit-learn (`sklearnex`) & oneDAL

### 7.1 How `sklearnex` Works Under the Hood

Standard Python Scikit-learn algorithms are largely single-threaded or rely on basic OpenMP loops with significant Python C-API overhead. **Intel oneDAL (oneAPI Data Analytics Library)** is a compiled C++ library utilizing AVX-512 vectorization, thread pools, and cache blocking.

### 7.2 Dynamic Monkey-Patching

By inserting two lines of code at the top of any script:

```
from sklearnex import patch_sklearn
patch_sklearn()
```

The `sklearnex` loader dynamically intercepts standard Scikit-learn module imports (`sklearn.cluster.KMeans`, `sklearn.ensemble.RandomForestClassifier`, `sklearn.svm.SVC`) and substitutes them with accelerated Intel oneDAL implementations with **zero changes to model parameters or training logic**.

### 7.3 Measured Acceleration Factors

- **K-Means Clustering**: 15× to 35× speedup.
- **Principal Component Analysis (PCA)**: 10× to 25× speedup.

- **Support Vector Classification (SVC)**: 20× to 80× speedup.
- **Random Forest Training**: 8× to 15× speedup.

---

## Module 8: Performance Benchmarking & Scaling Math

### 8.1 Performance Metrics

- **Speedup**: $S = \frac{T_{\text{baseline}}}{T_{\text{accelerated}}}$
- **Parallel Efficiency**: $E = \frac{S}{N_{\text{cores}}} \times 100\%$
- **Inference Throughput**: $\text{Throughput} = \frac{\text{Total Samples}}{\text{Latency in Seconds}} \quad (\text{samples/sec})$

---

**Study Guide Status:** 100% Complete & Verified | GLA University HPC Masterclass Day 2